

Ex No: 1	Creating a new Git repository, cloning an existing repository, checking changes, pushing changes to a Git remote and creating a Git branch
Date :	

Aim:

To create and manage a Git repository by initializing a repository, cloning an existing repository, checking changes, staging and committing files, pushing changes to a remote repository, and creating a new Git branch.

Software / Requirements:

- Git
- Command Prompt / PowerShell / Linux Terminal
- A Git remote repository (for push operation)

Algorithm / Procedure:

1. Creating a New Git Repository

Open a terminal or command prompt.
 Navigate to the folder where the project has to be created.
 Initialize a Git repository using git init.
 Create a sample file such as README.md.

2. Checking Repository Status

Use git status to view untracked, modified and staged files.
 Observe the changes reported by Git.

3. Configuring Git

Configure the user name and email if Git has not already been configured.
 These details are recorded with commits.

4. Staging and Committing Changes

Stage the required file using git add or stage all files.
 Commit the staged changes with a meaningful commit message.

5. Cloning an Existing Repository

Use git clone followed by the repository URL.
 Move into the cloned directory and inspect the project files.

6. Pushing Changes to a Remote

Connect the local repository to a remote repository if required.
 Push the committed changes to the required branch.

7. Creating a Git Branch

Create a new branch and switch to it.
 Make a small change, commit it, and verify the active branch.

OUTPUT:

```
C:\> git --version
git version 2.46.0

C:\> mkdir devops-git
C:\> cd devops-git
C:\devops-git> git init
Initialized empty Git repository in C:/devops-git/.git/

C:\devops-git> echo "# DevOps Git Lab" > README.md

C:\devops-git> git status
On branch master

No commits yet

Untracked files:
  README.md

C:\devops-git> git config --global user.name "Your Name"
C:\devops-git> git config --global user.email "you@example.com"

C:\devops-git> git add README.md
C:\devops-git> git commit -m "Initial commit"
[master (root-commit) a1b2c3d] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md

C:\devops-git> git clone <REPOSITORY-URL>
Cloning into '<REPOSITORY-NAME>'...
remote: Enumerating objects...
Receiving objects: 100%...
done.

C:\devops-git> git remote -v
origin <REMOTE-URL> (fetch)
origin <REMOTE-URL> (push)

C:\devops-git> git branch -M main
C:\devops-git> git push -u origin main
Enumerating objects: 3, done.
Writing objects: 100% (3/3), done.
branch 'main' set up to track 'origin/main'.

C:\devops-git> git switch -c feature-demo
Switched to a new branch 'feature-demo'

C:\devops-git> git branch
* feature-demo
  main

C:\devops-git> git status
On branch feature-demo
nothing to commit, working tree clean
```

On completion of the above program the following PO's and PSO's are attained

Course Outcomes	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2	PSO 1	PSO 2	PSO 3
P405.1	3	3	3	3	3	2	2	2	3	3	3	3	3	2	3

Result:

Thus the Git repository was created, an existing repository was cloned, changes were checked, committed and pushed to a remote repository, and a new Git branch was created and verified successfully.

Ex No: 2	Installing Maven and setting up a simple Java project with pom.xml
Date :	

Aim:

To install Apache Maven, create a simple Java project with a pom.xml file, and build the project using the compile, test and package commands.

Software / Requirements:

- JDK
- Apache Maven
- Command Prompt / PowerShell / Linux Terminal

Algorithm / Procedure:**1. Verify Java Installation**

Install a JDK and verify that the java and javac commands are available.

Set JAVA_HOME if required by the operating system.

2. Install Maven

Download and extract Maven.

Add the Maven bin directory to the system PATH.

Verify the installation using mvn -version.

3. Create the Project Structure

Create a Maven project directory.

Create src/main/java and src/test/java folders.

Create a pom.xml file in the project root.

4. Create the Java Application

Create a simple HelloWorld Java class.

Write a main method that prints a greeting message.

5. Build the Project

Run mvn compile to compile the source code.

Run mvn test to execute tests.

Run mvn package to compile, test and create the packaged JAR.

OUTPUT:

```
C:\> java -version
java version "17.x.x"

C:\> javac -version
javac 17.x.x

C:\> mvn -version
Apache Maven 3.x.x
Maven home: C:\apache-maven
Java version: 17.x.x
OS name: Windows

C:\> mkdir MavenHello
C:\> cd MavenHello

C:\MavenHello> mkdir src\main\java\com\example
C:\MavenHello> mkdir src\test\java\com\example

C:\MavenHello> mvn compile
[INFO] Scanning for projects...
[INFO] Compiling source files...
[INFO] BUILD SUCCESS
[INFO] Total time: 2.1 s

C:\MavenHello> mvn test
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

C:\MavenHello> mvn package
[INFO] Building jar: target\maven-hello-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS

C:\MavenHello> mvn clean package
[INFO] Cleaning target directory...
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building jar: target\maven-hello-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
```

On completion of the above program the following PO's and PSO's are attained

Course Outcomes	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2	PSO 1	PSO 2	PSO 3
P405.1	3	3	3	3	3	2	2	2	3	3	3	3	3	2	3

Result:

Thus Maven was installed and verified. A simple Java project containing pom.xml was created and successfully built using the compile, test and package commands.

Ex No: 3	Installing Docker container on Windows/Linux and issuing Docker commands
Date :	

Aim:

To install Docker on Windows/Linux and execute basic Docker commands for downloading images, creating containers, viewing containers and images, and managing Docker resources.

Software / Requirements:

- Docker Desktop (Windows) or Docker Engine (Linux)
- Command Prompt / PowerShell / Linux Terminal

Algorithm / Procedure:

1. Installing Docker on Windows

Install Docker Desktop and complete the installation.

Start Docker Desktop and wait until the Docker engine is running.

Open PowerShell or Command Prompt and verify the installation.

2. Installing Docker on Linux

Install Docker Engine using the package manager or the official installation method for the selected Linux distribution.

Start and enable the Docker service.

Verify the Docker version.

3. Pulling an Image

Download a small official image such as hello-world or ubuntu.

List the locally available images.

4. Creating and Running a Container

Run a container from the selected image.

Use an interactive terminal with Ubuntu if required.

List running and stopped containers.

5. Container Management

Stop a running container.

Remove the stopped container.

Remove an image when it is no longer required.

OUTPUT:

```
C:\> docker --version
Docker version 27.x.x, build xxxxxxxx
```

```
C:\> docker info
Client:
  Version:      27.x.x
  Context:      desktop-linux
```

```
Server:
  Containers: 0
  Images: 0
  Operating System: Docker Desktop
```

```
C:\> docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
Status: Downloaded newer image for hello-world:latest
```

```
C:\> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-world	latest	xxxxxxx	2 weeks ago	10kB

```
C:\> docker run hello-world
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

```
C:\> docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Status: Downloaded newer image for ubuntu:latest
```

```
C:\> docker run -it ubuntu
root@xxxxxxx:/#
```

```
C:\> docker ps
```

CONTAINER ID	IMAGE	COMMAND	STATUS
xxxxxxxxxxx	ubuntu	"/bin/bash"	Up 10 seconds

```
C:\> docker ps -a
```

CONTAINER ID	IMAGE	STATUS
xxxxxxxxxxx	ubuntu	Exited
yyyyyyyyyyy	hello-world	Exited

```
C:\> docker stop <CONTAINER_ID>
<CONTAINER_ID>
```

```
C:\> docker rm <CONTAINER_ID>
<CONTAINER_ID>
```

```
C:\> docker rmi <IMAGE_ID>
Untagged: ubuntu:latest
Deleted: sha256:xxxxxxx
```

On completion of the above program the following PO's and PSO's are attained

Course Outcomes	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
P405.1	3	3	3	3	3	2	2	2	3	3	3	3	3	2	3

Result:

Thus Docker was installed and verified successfully. Docker images were pulled, containers were created and executed, and basic Docker management commands were performed successfully.

Ex No: 4	Creating and executing an Ansible YAML playbook to automate tasks.
Date :	

Aim:

To write a simple YAML file containing variables and tasks in Ansible, execute the playbook, and observe how Ansible automates system tasks.

Software / Requirements:

- Linux system or WSL2
- Python 3
- Ansible
- Text editor / Terminal

Algorithm / Procedure:

1. Install and Verify Ansible

Open a Linux terminal or WSL environment.

Install Ansible using the system package manager or Python package manager.

Verify the installation using `ansible --version`.

2. Create an Inventory File

Create an inventory file named `hosts.ini`.

For a simple local demonstration, define `localhost` as the managed host.

3. Create the YAML Playbook

Create a file named `setup.yml`.

Define variables and tasks in YAML format.

Use tasks to create a directory, create a text file and display a message.

4. Run the Playbook

Execute the playbook with `ansible-playbook`.

Observe the `PLAY`, `TASK` and `changed/ok` status messages.

Run the playbook again and observe that already completed tasks can become idempotent.

OUTPUT:

```
$ ansible --version
ansible [core 2.x.x]
  python version = 3.x.x

$ ansible-inventory -i hosts.ini --list
{
  "_meta": {
    "hostvars": {
      "localhost": {
        "ansible_connection": "local"
      }
    }
  },
  "all": {
    "children": ["ungrouped"]
  }
}

$ ansible-playbook -i hosts.ini setup.yml

PLAY [Simple Ansible Automation] *****

TASK [Create project directory] *****
changed: [localhost]

TASK [Create message file] *****
changed: [localhost]

TASK [Display completion message] *****
ok: [localhost] => {
  "msg": "Ansible automation completed"
}

PLAY RECAP *****
localhost                : ok=3    changed=2    unreachable=0
                          failed=0    skipped=0    rescued=0
                          ignored=0

$ cat /tmp/devops_lab/message.txt
Ansible automation completed

$ ansible-playbook -i hosts.ini setup.yml

PLAY RECAP *****
localhost                : ok=3    changed=0    unreachable=0
                          failed=0    skipped=0
```

On completion of the above program the following PO's and PSO's are attained

Course Outcomes	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2	PSO 1	PSO 2	PSO 3
P405.1	3	3	3	3	3	2	2	2	3	3	3	3	3	2	3

Result:

Thus the YAML playbook was created and executed successfully. The defined variables and tasks were processed by Ansible, and the automated changes were verified on the target system.

Ex No: 5	Setting up Docker and Maven in Jenkins and performing the first Pipeline Run
Date :	

Aim:

To configure Maven and Docker for Jenkins and create and execute a basic Jenkins Pipeline that checks out source code, builds a Maven project and builds a Docker image.

Software / Requirements:

- Jenkins
- JDK
- Apache Maven
- Docker
- Git

Algorithm / Procedure:

1. Install and Access Jenkins

Install Jenkins and start the Jenkins service.

Open the Jenkins dashboard using the configured local URL.

Complete the initial setup and install the required suggested plugins.

2. Configure Maven

Open Manage Jenkins and Global Tool Configuration.

Add a Maven installation or configure the existing Maven installation.

Give the Maven installation a name such as Maven3.

3. Configure Docker

Ensure Docker is installed and running on the Jenkins host.

Verify that the Jenkins service account has permission to execute Docker commands.

4. Create a Pipeline Job

Select New Item, choose Pipeline and provide a job name.

Enter a Pipeline script in the Pipeline section.

Save the job.

5. Run the Pipeline

Select Build Now.

Open Console Output and observe the stages.

Verify the Maven build and Docker image creation.

OUTPUT:

Started by user Student

[Pipeline] Start of Pipeline

[Pipeline] { (Checkout)

[Pipeline] git

Checking out source code...

> git checkout main

Checkout completed successfully

[Pipeline] }

[Pipeline] { (Maven Build)

[Pipeline] bat

C:\Jenkins\workspace\DevOpsPipeline> mvn clean package

[INFO] Scanning for projects...

[INFO] Compiling source files...

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[INFO] BUILD SUCCESS

[Pipeline] }

[Pipeline] { (Docker Build)

[Pipeline] bat

C:\Jenkins\workspace\DevOpsPipeline> docker build -t devops-hello:latest .

[+] Building...

Successfully built xxxxxxxx

Successfully tagged devops-hello:latest

[Pipeline] }

[Pipeline] End of Pipeline

Finished: SUCCESS

Pipeline Stages

Checkout SUCCESS

Maven Build SUCCESS

Docker Build SUCCESS

Docker image created:

devops-hello:latest

On completion of the above program the following PO's and PSO's are attained

Course Outcomes	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2	PSO 1	PSO 2	PSO 3
P405.1	3	3	3	3	3	2	2	2	3	3	3	3	3	2	3

Result:

Thus Maven and Docker were configured in Jenkins and the first Pipeline was created and executed successfully. The console output was observed and the build stages were verified.

Ex No: 6	Creating a GitHub account, repository and pushing a simple Hello World application
Date :	

Aim:

To create a GitHub account and repository, prepare a simple application, connect the local Git repository to GitHub, and push the application source code.

Software / Requirements:

- Web browser
- Git
- GitHub account
- Command Prompt / PowerShell / Linux Terminal

Algorithm / Procedure:

1. Create a GitHub Account

Open GitHub in a web browser.

Create a new account or sign in to an existing account.

2. Create a Repository

Select New repository.

Provide a repository name such as hello-world-devops.

Choose the required visibility and create the repository.

3. Create a Simple Application

Create a local project directory.

Create a simple Hello World source file and README.md.

Initialize the directory as a Git repository.

4. Commit and Push to GitHub

Add the project files to the staging area.

Commit the files with a descriptive message.

Add the GitHub repository as the remote origin.

Push the main branch to GitHub.

5. Verify the Repository

Open the GitHub repository in a browser.

Confirm that the application files and README.md are visible.

OUTPUT:

```
C:\> mkdir hello-world-devops
C:\> cd hello-world-devops

C:\hello-world-devops> echo "# Hello World DevOps Project" > README.md

C:\hello-world-devops> git init
Initialized empty Git repository in C:/hello-world-devops/.git/

C:\hello-world-devops> git add .
C:\hello-world-devops> git commit -m "Add Hello World application"

[master (root-commit) alb2c3d] Add Hello World application
 2 files changed
 create mode 100644 README.md
 create mode 100644 HelloWorld.java

C:\hello-world-devops> git branch -M main

C:\hello-world-devops> git remote add origin https://github.com/<USERNAME>/hello-world-devops.git

C:\hello-world-devops> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Writing objects: 100% (4/4), done.
branch 'main' set up to track 'origin/main'.

GitHub Repository:
https://github.com/<USERNAME>/hello-world-devops

Repository contents:
README.md
HelloWorld.java

HelloWorld.java execution:
Hello World
```

On completion of the above program the following PO's and PSO's are attained

Course Outcomes	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2	PSO 1	PSO 2	PSO 3
P405.1	3	3	3	3	3	2	2	2	3	3	3	3	3	2	3

Result:

Thus a GitHub account and repository were created, a simple Hello World application was committed to the local Git repository, pushed to GitHub, and verified successfully.

Ex No: 7	Building a simple application using Gradle
Date :	

Aim:

To install and configure Gradle, create a simple Java application using Gradle, define the build configuration, and build and run the application.

Software / Requirements:

- JDK
- Gradle
- Command Prompt / PowerShell / Linux Terminal

Algorithm / Procedure:

1. Verify Java and Gradle

Install a JDK and verify java -version.

Install Gradle or use the Gradle Wrapper generated for the project.

Verify the Gradle installation using gradle -version.

2. Create the Project

Create the Gradle project directory and source folder.

Create settings.gradle and build.gradle.

Create a simple Java application class.

3. Configure build.gradle

Apply the Java plugin.

Define the project group and version.

Configure the application main class.

4. Build the Application

Run gradle clean.

Run gradle build to compile and test the application and generate the JAR.

Run gradle run to execute the application.

5. Verify the Output

Observe the build result.

Check the build/libs directory for the generated JAR file.

OUTPUT:

```
C:\> gradle -version

-----
Gradle 8.x
-----

Build time:    202x-xx-xx
Kotlin:        1.x.x
Groovy:        3.x.x
JVM:           17.x.x
OS:            Windows

C:\gradle-hello> gradle clean

BUILD SUCCESSFUL in 1s
1 actionable task: 1 executed

C:\gradle-hello> gradle build

> Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
> Task :jar
> Task :assemble
> Task :compileTestJava
> Task :test
> Task :check
> Task :build

BUILD SUCCESSFUL in 4s
7 actionable tasks: 7 executed

C:\gradle-hello> gradle run

> Task :run
Hello from Gradle!

BUILD SUCCESSFUL in 2s

C:\gradle-hello> gradle jar

BUILD SUCCESSFUL in 1s

Generated JAR:
build\libs\gradle-hello-1.0.jar
```

On completion of the above program the following PO's and PSO's are attained

Course Outcomes	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2	PSO 1	PSO 2	PSO 3
P405.1	3	3	3	3	3	2	2	2	3	3	3	3	3	2	3

Result:

Thus a simple Java application was created using Gradle, the build configuration was defined, the application was compiled and packaged successfully, and the program output was verified.